

Relatório de Auditoria de Segurança — GymFlow

Investigação Exaustiva de Vulnerabilidades de Código-Fonte, Arquitetura Multi-Tenant, Permissões Client-Side, IDOR, Segredos Hardcoded e Resistência a Injeções.

Projeto Avaliado:	GymFlow (Sistema de Gestão de Academias & Personal Trainers)
Data da Auditoria:	14 de Setembro de 2026
Classificação:	Repositório Público GitHub (cauacreis/GymFlow)
Stack Tecnológica:	Next.js 14 (App Router), React 18, TypeScript, Tailwind CSS, Supabase (Auth, Postgres RLS, Storage), Mercado Pago API, Capacitor Mobile, Vercel Edge.
Escopo Auditado:	100% dos Handlers de Rotas API (src/app/api), Schemas e Migrações SQL (supabase/), Bibliotecas de Serviços (src/lib/), Stores Locais e Componentes de Interface (src/components/).

Nota Metodológica & Mapeamento para a Stack GymFlow:

A auditoria investigou 5 categorias fundamentais, mapeadas especificamente para a arquitetura detectada:

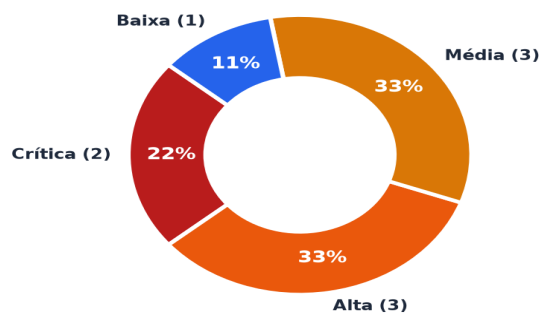
- Banco sem Tranca:** Investigou políticas de Row Level Security (RLS) no Supabase e queries de listagem sem filtro pelo UUID autenticado.
- Permissão Definida no Navegador:** Confrontou gates de permissão (activeRole, isAdmin, planTier) no frontend com a ausência de validação correspondente nas mutações do backend.
- IDOR (Insecure Direct Object Reference):** Mapeou endpoints e chamadas que consultam ou alteram entidades (students, bookings, payments) sem checar posse.
- Chaves Expostas (Hardcode):** Varreu segredos, salts criptográficos, defaults perigosos e fallbacks silenciosos no código e no histórico git.
- Inputs sem Tratamento (XSS):** Auditou renderizações React (dangerouslySetInnerHTML, URLs javascript:, eval) e sanitização de dados no cliente e no servidor.

1. Resumo Executivo

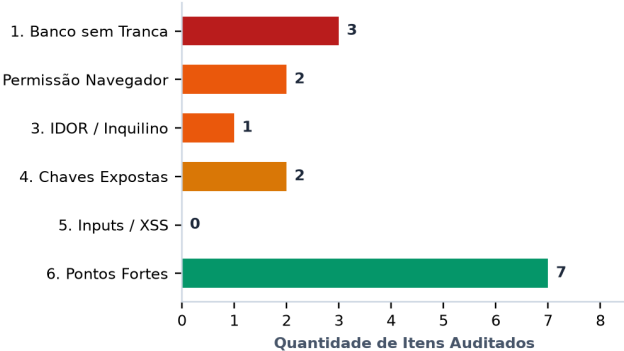
A auditoria no GymFlow identificou **9 achados verificados** no código-fonte real: **2 Críticas**, **3 Altas**, **3 Médias** e **1 Baixa**. Também foram verificados e comprovados **7 pontos fortes arquiteturais**, demonstrando que proteções avançadas (como FSM monotônica em webhooks e bloqueio de adulteração de preços) já estão ativas, coexistindo com políticas permissivas no banco.

CRÍTICA 2	ALTA 3	MÉDIA 3	BAIXA 1	PONTOS FORTES 7
----------------------------	-------------------------	--------------------------	--------------------------	----------------------------------

Achados por Severidade



Distribuição por Categoria & Pontos Fortes



2. Pontos Fortes & Riscos Centrais

■ Pontos Fortes Auditados e Comprovados:

- **Prevenção de Adulteração de Preços (pix/preference/subscription routes):** Preços validados estritamente no servidor via OFFICIAL_PLANS.
- **FSM Monotônica em Webhooks (webhook/route.ts & payment/check/route.ts):** resolvePaymentStateTransition impede regressão de approved.
- **Trigger PostgreSQL Anti-Tampering (schema.sql & 20260911000100):** protect_profile_subscription_fields() bloqueia updates anon em planos.
- **Proteção contra Open Redirect (auth/callback/route.ts):** Validação de caminho estrito em next impede links maliciosos.
- **Resistência Nativa a XSS:** React 18 JSX default escaping, zero dangerouslySetInnerHTML, sem eval ou markdown não sanitizado.
- **Rate Limiting no Edge / Middleware:** Janelas deslizantes em /api/auth, /api/payment e /api/vault.
- **Pre-Commit Script & Gitignore:** scripts/check-secrets.js bloqueia staging de credenciais APP_USR- e TEST-.

■ Riscos Centrais Identificados:

- **RLS Aberto FOR ALL USING (true):** Anon key pública tem permissão irrestrita de leitura, escrita e deleção em quase todas as tabelas.
- **Salt do Cofre Hardcoded no GitHub Público:** VAULT_SALT = 'gymflow_vault_salt_2026' permite forjar sessões super admin HMAC.
- **Vazamento Massivo no Carregamento:** Stores sincronizam todas as tabelas sem filtro de coach/aluno salvando no localStorage de visitantes.

3. Tabela Detalhada de Achados por Categoria

Todos os 9 achados verificados no código real, mapeados por severidade, arquivo e linha:

Sev.	Categoria	Arquivo : Linhas	Descrição do Risco & Explorabilidade
CRÍTICA	1. Banco sem Tranca	supabase/schema.sql:223-242 migrations/init.sql:146-165	Políticas RLS com FOR ALL USING (true) em profiles, students, bookings e workouts. Qualquer cliente anônimo pode ler, sobrescrever ou deletar qualquer registro.
CRÍTICA	4. Chaves Expostas	api/vault/verify/route.ts:8 api/vault/session/route.ts:7	Salt VAULT_SALT = 'gymflow_vault_salt_2026' hardcoded no GitHub público. Atacantes podem forjar tokens de sessão HMAC para a cookie gymflow_vault_auth e desbloquear o cofre.
ALTA	1. Banco sem Tranca	lib/workout-store.ts:396 lib/booking-store.ts:928	Sincronização assíncrona automática chama fetchStudentsFromSupabase() sem filtrar tenant/coach, despejando a lista completa de alunos e contatos no localStorage do navegador.
ALTA	2. Permissão Navegador	lib/auth-store.ts:128,251 components/coach/*	Contas têm enabledRoles: ['student', 'coach'] por padrão. Qualquer aluno pode alternar para 'coach' no cliente e acessar o painel de treinador sem validação no servidor.
ALTA	3. IDOR / Inquilino	api/payment/check/route.ts:73,125	O parâmetro query userId sobrepõe o parsedRef.userId do pagamento aprovado. Permite que um atacante atribua um pagamento alheio ao seu próprio UUID.
MÉDIA	4. Chaves Expostas	lib/supabase.ts:113 auth/callback/route.ts:39	Fallback silencioso SUPABASE_SERVICE_ROLE_KEY supabaseAnonKey. Em caso de ausência da chave, rotas administrativas executam como cliente anon sem lançar erro.
MÉDIA	4. Chaves Expostas	api/payment/webhook/route.ts:45	Validação HMAC encapsulada em if (webhookSecret && signatureHeader). Requisições sem o header x-signature pulam a checagem sem retornar 401.
MÉDIA	2. Permissão Navegador	api/delete-account/route.ts:12 api/export-data/route.ts:7	Rotas de conformidade LGPD não extraem nem verificam o token de sessão do chamador. delete-account apenas apaga um cookie estático sem tocar no banco.
BAIXA	1. Banco sem Tranca	api/auth/check-device/route.ts:40	Endpoint público informa explicitamente se um e-mail já existe na tabela de perfis ('Este e-mail já está em uso'), permitindo enumeração de clientes.

4. Recomendações Priorizadas de Correção

A matriz de remediação deve ser executada na seguinte ordem de prioridade técnica:

P1 — Urgente (Imediato)

- **Corrigir Políticas RLS no Supabase:** Substituir as políticas USING (true) por verificações baseadas em `auth.uid() = user_id` ou `auth.uid()::text = coach_id`. Desabilitar mutações diretas anônimas.
- **Eliminar Chaves e Salts Hardcoded no Vault:** Mover `VAULT_SALT` e a chave mestra para variáveis de ambiente seguras (`ADMIN_VAULT_SECRET` e `ADMIN_MASTER_KEY`). Rejeitar inicialização se não configuradas.

P2 — Alta Prioridade

- **Sanitizar Métodos de Sincronização Local:** Exigir `coachId` ou `userId` em `fetchStudentsFromSupabase` e `fetchBookingsFromSupabase`. Nunca disparar queries sem filtro.
- **Blindar o Atribuidor de Pagamento em /api/payment/check:** Remover a precedência de `userId` via query param. O usuário deve ser extraído estritamente do `external_reference` ou sessão autenticada.
- **Isolar Papéis de Treinador e Aluno no Servidor:** Mudar cadastro para `enabled_roles: [role]` e criar endpoints dedicados para ações de professor com verificação de papel no backend.

P3 — Média Prioridade

- **Exigir Assinatura Obrigatória no Webhook:** Rejeitar com 401 qualquer requisição sem o header `x-signature` ou com assinatura inválida.
- **Falha Explícita em Variáveis de Ambiente:** Remover fallback silencioso `SUPABASE_SERVICE_ROLE_KEY || supabaseAnonKey`, lançando exceção no boot.
- **Autenticar Endpoints LGPD:** Validar a sessão do chamador antes de processar pedidos de exportação e exclusão de dados.

5. Issues Prontas para o GitHub

Copie e cole os blocos abaixo diretamente na aba **Issues** do repositório GitHub para criar as tarefas com checklist verificável:

--- ISSUE 1 ---

Título: [Segurança] Corrigir RLS com políticas abertas USING (true) nas tabelas operacionais

Labels: security, severidade:crítica

Descrição do Problema & Explorabilidade:

As políticas RLS de profiles, students, coach_plans, student_workouts, bookings e device_registrations estão configuradas com FOR ALL USING (true). Qualquer cliente com a chave anônima (pública no frontend) pode ler, alterar e excluir dados de qualquer usuário ou treinador da academia.

Evidência Técnica:

supabase/schema.sql linhas 223-242; supabase/migrations/20260911000000_init.sql linhas 146-165.

Impacto:

Vazamento em massa de dados confidenciais (LGPD), adulteração de treinos e exclusão maliciosa de agendamentos e cadastros.

Sugestão de Correção:

Reescrever as políticas RLS para checar `auth.uid() = user\_id` em dados de alunos e vincular os acessos de treinadores estritamente ao seu perfil correspondente. Negar mutações diretas anônimas.

Critérios de Aceite:

- [] SELECT na tabela 'students' retorna apenas os alunos vinculados ao coach_id do usuário autenticado
- [] SELECT na tabela 'bookings' restringe registros onde auth.uid() coincide com student_id ou coach_id
- [] Teste de mutação (UPDATE/DELETE) com cliente anon em registro alheio é rejeitado pelo PostgreSQL

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

Título: [Segurança] Eliminar segredo criptográfico hardcoded VAULT_SALT e blindar autenticação do cofre

Labels: security, severidade:crítica

Descrição do Problema & Explorabilidade:

A constante VAULT_SALT = 'gymflow_vault_salt_2026' está hardcoded no repositório público GitHub. Qualquer atacante pode gerar tokens de sessão válidos no formato payload.signature usando HMAC-SHA256 e obter acesso irrestrito ao painel de administração em /admin-vault.

Evidência Técnica:

src/app/api/vault/verify/route.ts linhas 8-11; src/app/api/vault/session/route.ts linhas 7-8; src/lib/admin-vault.ts linhas 162-164.

Impacto:

Bypass completo do cofre administrativo, permitindo exportar backup integral do banco, manipular alunos e alterar configurações globais.

Sugestão de Correção:

Mover VAULT_SALT para a variável de ambiente segura ADMIN_VAULT_SECRET. Armazenar a chave mestra exclusivamente em variável de ambiente (ADMIN_MASTER_KEY). Eliminar a validação ingênua baseada em presença de cookie sem validação de assinatura no cliente.

Critérios de Aceite:

- [] Ausência de strings de segredo ou hash estático no código-fonte
- [] Cookie gymflow_vault_auth assinado com segredo oriundo de process.env
- [] Falsificação manual de cookie sem assinatura válida resulta em 401 imediato

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

Título: [Segurança] Eliminar queries globais sem filtro de tenant no sincronizador de cache

Labels: security, severidade:alta

Descrição do Problema & Explorabilidade:

O carregamento de stores locais chama `fetchStudentsFromSupabase()` e `fetchBookingsFromSupabase()` sem passar parâmetros de identificação, solicitando todos os registros do banco e salvando-os no `localStorage` do visitante.

Evidência Técnica:

`src/lib/workout-store.ts` linha 396; `src/lib/booking-store.ts` linha 928; `src/lib/supabase-service.ts` linhas 20-24, 186-190.

Impacto:

Exposição de dados cadastrais, telefones e rotinas de todos os alunos para qualquer usuário conectado ao aplicativo.

Sugestão de Correção:

Condicionar a sincronização de cache à presença de um `currentUser` autenticado e repassar obrigatoriamente o `coach_id` ou `student_id`.

Critérios de Aceite:

[] `fetchStudentsFromSupabase` só executa se `coachId` for uma string válida e não vazia
[] Inspeccionar `localStorage` no DevTools de um aluno e confirmar que nenhum dado de outros alunos é armazenado
--- FIM ISSUE 3 ---

--- ISSUE 4 ---

Título: [Segurança] Proteger endpoint `/api/payment/check` contra IDOR e usurpação de crédito

Labels: security, severidade:alta

Descrição do Problema & Explorabilidade:

A rota aceita o parâmetro `userId` via query string e aplica o crédito de 30 dias com base nele com prioridade sobre o `external_reference`. Um atacante pode informar o ID de pagamento de outra pessoa e passar seu próprio `userId`.

Evidência Técnica:

`src/app/api/payment/check/route.ts` linhas 32, 73-74, 125-151.

Impacto:

Fraude financeira e roubo de tempo de assinatura entre contas.

Sugestão de Correção:

Extrair o usuário beneficiário estritamente de `parsedRef.userId` e exigir que o usuário chamador esteja autenticado na sessão Supabase correspondente.

Critérios de Aceite:

[] `userId` passado como query param não pode sobrescrever o dono do pagamento gravado no `external_reference`
[] Chamadas sem sessão correspondente ou divergentes são rejeitadas com 403 Forbidden
--- FIM ISSUE 4 ---

--- ISSUE 5 ---

Título: [Segurança] Tornar validação de assinatura HMAC obrigatória no webhook do Mercado Pago

Labels: security, severidade:média

Descrição do Problema & Explorabilidade:

O webhook avalia a assinatura HMAC apenas se o cabeçalho `x-signature` for enviado. Caso omitido, o código segue fluxo sem validar autenticidade da notificação.

Evidência Técnica:

`src/app/api/payment/webhook/route.ts` linhas 45-60.

Impacto:

Risco de spoofing de eventos e esgotamento do ledger de eventos de webhook.

Sugestão de Correção:

Rejeitar imediatamente com 401 Unauthorized qualquer requisição sem o header `x-signature` e sem chave de segredo configurada.

Critérios de Aceite:

[] POST em `/api/payment/webhook` sem header `x-signature` retorna HTTP 401
[] POST com assinatura forjada retorna HTTP 401
[] Notificações oficiais do Mercado Pago continuam sendo processadas com sucesso
--- FIM ISSUE 5 ---